



Selected Topics in Software Engineering

Campus Food Ordering System

Team Members:

Jana AlHuthayli 444002015

Jory Alkhudari 444005065

Norah Alkabkabi 444003676

Arwa Al Hassani 44411717

Jana Alotaibi 444007448

Group Members: 1

Instructor Name: Dr. Afnan Aldhahri

Submission Date: 06/06/2026



Table of contents

1. System Overview
 - 1.1. Problem Description
 - 1.2. System Objectives
 - 1.3. Main Features
 - 1.4. Users and Roles
2. Microservices Design
 - 2.1. System Decomposition (Microservices Identification)
 - 2.2. Data Management per Service
 - 2.3. System Architecture Diagram
 - 2.4. External Systems Integration
 - 2.5. Technology Stack Selection & Justification
3. DevOps Strategy
 - 3.1. CI/CD Pipeline Overview
 - 3.2. CI/CD Pipeline Diagram
 - 3.3. Pipeline Stages Explanation
 - 3.4. CI/CD Tool Selection (GitHub Actions / GitLab CI / Jenkins)
 - 3.5. Testing Strategy
4. AI Features
 - 4.1. Smart Food Recommendations
 - 4.2. Predicting Busy Times (Demand Forecasting)
 - 4.3. Ethical Considerations
5. Sustainability & Green Software
 - 5.1. Sustainability Overview
 - 5.2. Five Sustainability Dimensions
6. References
7. Team Contribution Statement



1. System Overview

1.1. Problem Description

University students frequently rely on campus cafeterias for their daily meals. During peak periods, such as lunch breaks and class transitions, cafeterias often experience long queues, overcrowding, and delayed service. Traditional ordering methods based on manual order taking and cash transactions can result in order inaccuracies, inefficient queue management, and limited visibility into food availability and order status [1], [9].

In addition, cafeteria managers often lack access to real-time operational data, making it difficult to forecast demand and allocate resources efficiently. As students increasingly expect digital and mobile-friendly services, there is a growing need for a modern ordering platform. The proposed Campus Food Ordering System addresses these challenges by allowing students to browse menus, place orders in advance, track order status, and receive AI-powered recommendations and busy-time predictions, while helping cafeteria staff manage operations more effectively [2], [3].

Intended Users

- Students
- Cafeteria Staff
- Cafeteria Managers
- System Administrators



1.2. System Objectives

1. **Digitize the ordering process:** Replace manual queue-based ordering with an online platform that allows students to place orders from anywhere on campus before arriving at the cafeteria.
2. **Reduce wait times and congestion:** Support advance ordering and pickup time slots so students spend less time in physical queues, especially during peak hours.
3. **Improve order accuracy and transparency:** Provide clear digital menus, order summaries, and real-time status updates (e.g., received, preparing, ready for pickup) to minimize errors and uncertainty.
4. **Support efficient cafeteria operations:** Give cafeteria staff and administrators tools to manage menus, orders, and operational visibility (e.g., order volume by time period).
5. **Enable secure and convenient access:** Authenticate users by role (student, staff, admin) and protect order and account data using appropriate security practices.
6. **Leverage AI for a better experience:** Use historical data to offer smart food recommendations and demand forecasting, following approaches used in AI-enabled food-ordering systems [3].
7. **Adopt a scalable technical foundation:** Implement the system using a microservices architecture, containerization, and CI/CD so services can evolve and scale independently [4].
8. **Align with sustainability and ethics goals:** Support efficient resource use, responsible AI use, and user privacy protection as defined in the ethical considerations section of this report.



1.3. Main Features

1.3.1. Ordering

- **Digital menu browsing:** Students view cafeteria menus with categories, prices, availability, and item details.
- **Cart and checkout:** Add items to a cart, apply notes (e.g., no spice), confirm order, and choose pickup time or “as soon as ready.”
- **Order tracking:** Real-time status: placed → confirmed → preparing → ready for pickup → completed.

1.3.2. Accounts

- **User registration and login :** Secure sign-in for students and staff; profile with contact info and preferences.
- **Order history:** View past orders for reordering and to support AI-based recommendations.

1.3.3. Payments

- **Campus-friendly payment:** Support for methods appropriate to the campus (e.g., cash at pickup, campus card, or online payment if integrated).

1.3.4. Cafeteria management

- **Menu management:** Staff and administrators add, update, or disable menu items and daily specials.
- **Order management dashboard:** Staff view incoming orders, update status, and manage the preparation queue.

1.3.5. Administration

- **User and role management :** Admins manage accounts, roles, and access permissions.
- **Reports and analytics :** Reports on orders, popular items, and peak periods to support operations and AI features.



1.3.6. AI

- **Smart food recommendations:** Personalized suggestions based on order history, preferences, and popularity patterns, similar to collaborative filtering approaches in university canteen systems [5], [6].
- **Predicting busy times:** Forecast peak demand periods so students can plan orders and cafeterias can prepare resources [3].

1.3.7. Technical

- **Microservices-based backend:** Separate services for users, menu, orders, payments, notifications, and AI/analytics [4].
- **REST APIs :** Standard APIs between services and clients.
- **Containerization and CI/CD:** Docker-based deployment and automated build, test, and deploy pipelines.

1.3.8. Quality

- **Testing strategy:** Unit, integration, and user acceptance testing to ensure reliability before release.



1.4. Users and Roles

Role	Main Actions
Student	Browse menus, place and pay for orders, track order status, view order history, receive AI recommendations, manage preferences
Cafeteria Staff	View and process incoming orders, update order status (preparing/ready), manage daily item availability, handle counter pickup
Cafeteria Manager / Vendor	Manage menus and pricing, monitor order flow, view operational reports, use peak-time forecasts for planning
System Administrator	Manage users and roles, configure system settings, monitor service health, oversee security and data policies
Guest (optional)	Limited browse-only access; redirected to register if campus policy requires a full account for ordering



2. Microservices Design

2.1. System Decomposition (Microservices Identification)

The Campus Food Ordering System is decomposed into five microservices. Each service owns a single bounded context, exposes its functionality through a REST API, and maintains its own dedicated database to ensure loose coupling and independent scalability.

Service	Responsibility	Data Managed	Communicates With
User Service	Handles user authentication, authorization, and account management.	User accounts, roles, credentials, preferences	All services (token validation); Notification Service (welcome/reset emails)
Menu Service	Manages cafeteria menu items, categories, pricing, and availability.	Menu items, categories, availability status, pricing	Order Service (item validation at checkout); AI/Analytics Service (item metadata for recommendations)
Order Service	Manages order placement, order lifecycle, recommendations, demand forecasting, and reporting.	Orders, order lines, order status, pickup times	Menu Service (validate items); Payment Service (trigger payment); Notification Service (status updates); AI/Analytics Service (order data feed)
Payment Service	Processes payment transactions and records payment status for orders.	Transactions, payment methods, payment status	Order Service (confirm payment result); external payment gateway
Notification Service	Sends system notifications for order updates and important events.	Notification logs, delivery status	Order Service (triggers); User Service (contact info lookup)
AI / Analytics Service	Provides basic recommendations and analytics based on order and menu data.	Order history aggregates, recommendation models, forecast outputs, report data	Order Service (consumes order events); Menu Service (item metadata); read-only access to aggregated user preference data



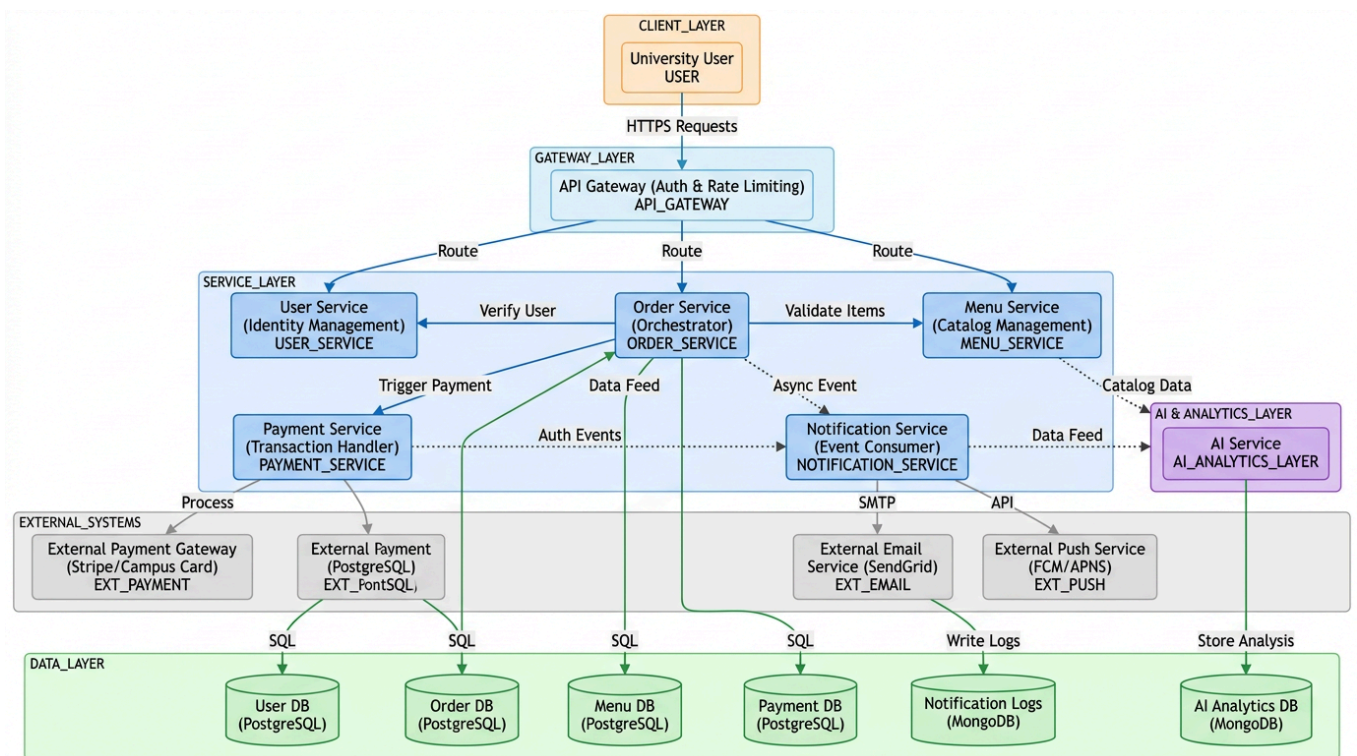
2.2. Data Management per Service

Each service owns its own datastore, following the database-per-service pattern to ensure that services can be deployed, scaled, and modified independently without shared schema dependencies [9].

Service	Database Type	Justification
User Service	PostgreSQL (relational)	User accounts and roles have well-defined relational structures with strict integrity requirements.
Menu Service	PostgreSQL (relational)	Menu items, categories, and pricing relationships are naturally tabular and benefit from transactional consistency.
Order Service	PostgreSQL (relational)	Orders involve structured relational data with strong consistency requirements across order lines and status transitions.
Payment Service	PostgreSQL (relational)	Financial transaction records require ACID compliance, auditability, and strict schema enforcement.
Notification Service	MongoDB (document)	Notification logs are schema-flexible, write-heavy, and do not require relational joins, making a document store appropriate.
AI / Analytics Service	MongoDB (document)	Aggregated behavioral data, recommendation outputs, and forecast results are variable in schema and suit a flexible document model.



2.3. System Architecture Diagram



Inter-service communication uses synchronous REST calls for request-response interactions (e.g., Order Service calling Menu Service to validate items) and asynchronous event publishing via RabbitMQ for event-driven flows (e.g., Order Service publishing an order-placed event consumed by the Notification and AI/Analytics services).



2.4. External Systems Integration

External System	Integrated With	Purpose
Payment Gateway	Payment Service	Processes card or campus account transactions securely outside the system boundary
Email Provider	Notification Service	Delivers transactional emails such as order confirmations and pickup-ready alerts
Push Notification Provider	Notification Service	Delivers real-time mobile push notifications for order status changes

2.5. Technology Stack Selection & Justification

Component	Choice	Justification
Programming Language	Node.js (JavaScript)	Lightweight, non-blocking I/O suits high-concurrency order and notification flows; large ecosystem of libraries for REST APIs and messaging; consistent language across all services reduces context switching for the team.
Databases	PostgreSQL (transactional services) and MongoDB (document services)	PostgreSQL provides ACID compliance for orders and payments where data integrity is critical; MongoDB provides schema flexibility for logs, recommendations, and analytics data that do not have a fixed structure.
API Type	REST	REST is well-understood, widely supported, and sufficient for the request-response and CRUD patterns used across all service interfaces; it requires less tooling overhead than GraphQL for a system of this scale.
Containerization	Docker	Docker enables consistent environments across development, staging, and production, supports independent deployment of each microservice, and integrates directly with the Kubernetes-based CI/CD pipeline.
Message Broker	RabbitMQ	Decouples event-driven communication between services (order events, notification triggers, analytics feeds) so that a failure in one consumer does not block the producing service.



3. DevOps Strategy

3.1. CI/CD Pipeline Overview

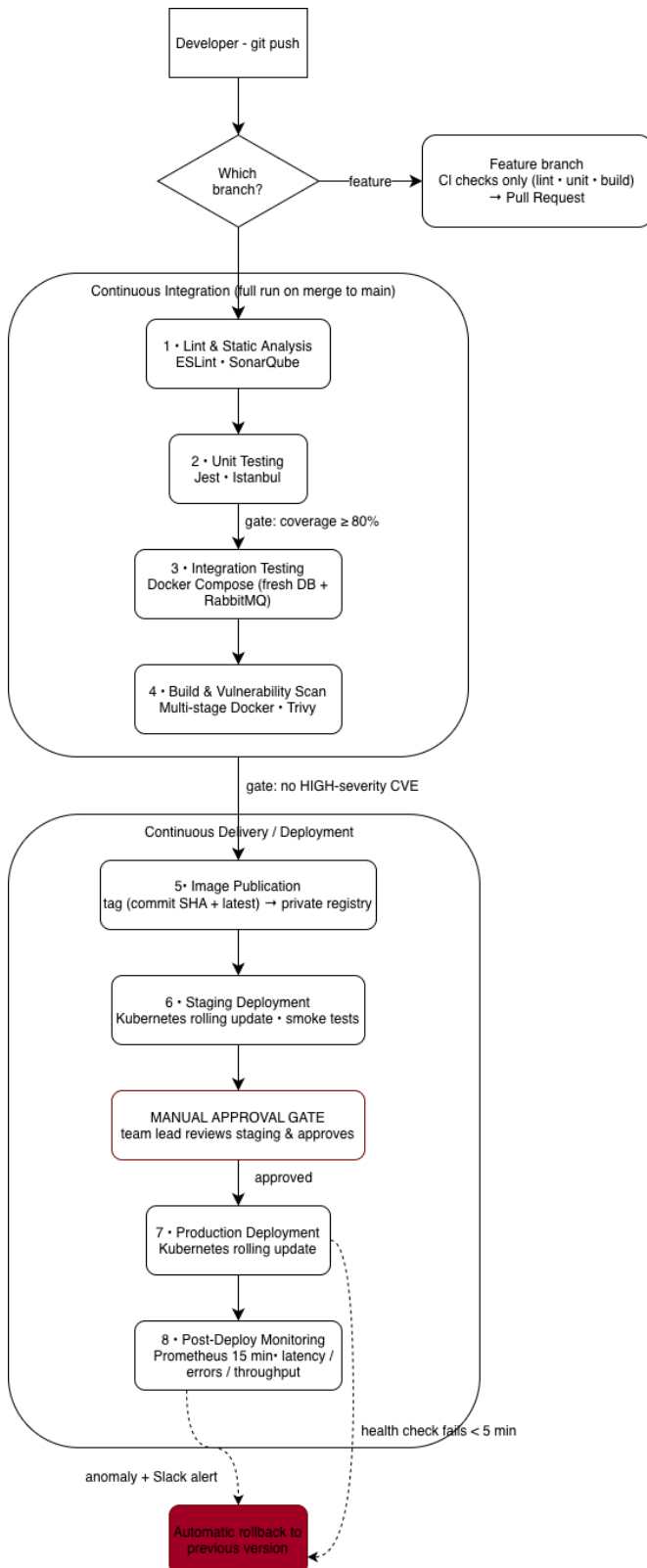
Our continuous integration and continuous deployment pipeline is fully automated. The pipeline takes over each time a developer pushes code: it runs tests, checks code quality, builds a Docker image, scans the Docker image for vulnerabilities, and deploys it to the staging environment. A designated team lead manually authorizes the deployment for production.

The objective is to shorten the time elapsed between the committing of a change and its appearance in production without sacrificing quality. When releases happen less often because deployment processes are manual, speed suffers, and risk increases. By deploying small changes often, we can catch problems as they arrive and rollback quickly. Thus, automated pipelines help us do this.

Each microservice has its own deployment definition, so we can deploy services independently.



3.2. CI/CD Pipeline Diagram





3.3. Pipeline Stages Explanation

Source control trigger. Pushing to a feature branch runs only the CI checks (lint, tests, build). Merging to the main branch runs the full pipeline including staging deployment, which keeps feature branches lightweight.

Lint and static analysis. ESLint enforces our code style and catches common JavaScript mistakes early. SonarQube goes deeper, looking for code smells, duplicated code, and potential security issues. A pull request can't be merged if SonarQube reports new critical problems.

Unit testing. Jest runs the unit tests for the service being built, and Istanbul measures code coverage. If coverage drops below 80% or any test fails, the pipeline stops.

Integration testing. A throwaway test environment is created with Docker Compose, including a fresh database and a temporary RabbitMQ instance. Integration tests then exercise real API endpoints with real database calls. This catches contract violations and configuration problems that unit tests miss.

Build and vulnerability scan. A multi-stage Dockerfile produces the smallest possible production image. Trivy scans the image against a CVE database, and the pipeline stops if it finds anything with a high severity rating.

Image publication. The image is tagged twice, once with the git commit SHA so we can trace exactly what code is running, and once with latest. Both tags are pushed to the private Docker registry.

Staging deployment. Kubernetes applies the new image to the staging cluster using a rolling update, which replaces one pod at a time so the service stays available throughout. Smoke tests then verify that the main user journeys still work end to end.

Production deployment with manual gate. This is the one stage that requires a human. A team lead reviews the staging results and approves the production deployment if everything looks healthy. Once approved, Kubernetes performs the same rolling update on production, with an automatic rollback if health checks fail within the first five minutes.

Post-deployment verification. Prometheus monitors latency, error rate, and throughput for fifteen minutes after each deployment. Sudden changes trigger an automatic rollback to the previous version and a Slack alert to the team.



3.4. CI/CD Tool Selection (GitHub Actions / GitLab CI / Jenkins)

Three industry-standard tools were considered: GitHub Actions, GitLab CI, and Jenkins.

Tool	Strengths	Trade-offs
GitHub Actions	Native to GitHub, where our code already lives; pull-request-triggered workflows map directly to our "feature branch = CI only, merge to main = full pipeline" model; large marketplace of ready-made actions (Node setup, Docker build/push, Trivy scan, kubectl); per-service workflow files suit independent deployment; no server to maintain.	Tied to GitHub; heavy custom logic can become verbose in YAML.
GitLab CI	Strong built-in CI with an integrated container registry and good Kubernetes support; single platform for repo, registry, and pipelines.	Requires hosting the project on GitLab; less convenient if the team already uses GitHub.
Jenkins	Most flexible and powerful; huge plugin ecosystem; fully self-hostable and tool-agnostic.	Must run and secure our own server, manage plugins and updates, and write more pipeline glue — significant operational overhead for a student team.

Selected tool: GitHub Actions. It aligns naturally with our branch-and-pull-request workflow, requires no self-managed CI server (unlike Jenkins), and integrates directly with the tools already in the pipeline through prebuilt actions. Storing a workflow file per service in the repository also matches our microservices approach, letting each service build, test, and deploy independently while keeping the configuration version-controlled alongside the code.



3.5. Testing Strategy

Testing is applied at three levels, each catching a different class of defect and running at a defined point in the pipeline.

3.5.1. Unit Testing

Jest tests individual functions and modules of a service in isolation, with external dependencies such as the database, RabbitMQ, and neighbouring services replaced by mocks so failures point clearly to the unit under test. Istanbul measures code coverage, and the pipeline stops if coverage falls below 80% or any test fails. These tests run on every push and give the fastest feedback. Typical targets include order-total calculation, input validation, and role-based access rules.

3.5.2. Integration Testing

Integration tests verify that a service works correctly with its *real* dependencies rather than mocks. A throwaway environment is created with Docker Compose, containing a fresh database and a temporary RabbitMQ instance, and the tests then exercise real API endpoints with real database calls and real message publishing/consuming. This level catches contract violations between services, schema mismatches, and configuration problems that unit tests cannot see. It runs in CI after unit tests pass and before the image is built.

3.5.3. User Acceptance Testing (UAT)

UAT validates the system against user requirements with representative users — students, cafeteria staff, and a manager — before a release is approved. It is performed on the staging environment using scenario-based scripts that follow real user journeys: browsing the menu, placing and paying for an order, tracking its status, staff updating an order through its lifecycle, and a manager viewing reports. Each scenario has clear acceptance criteria, and the testers' sign-off is the human checkpoint at the manual approval gate before production deployment. Feedback from UAT is logged and fed back to the development team for fixes prior to release.



4. AI Features

4.1. Smart Food Recommendations

4.1.1. What it does

This feature utilizes artificial intelligence techniques to analyze users' historical ordering behavior and preferences in order to generate personalized food recommendations. It suggests relevant menu items such as frequently ordered meals, similar dishes, and popular items among students with similar preferences[2],[6].

4.1.2. Benefits to users

This feature enhances the overall user experience by providing personalized meal suggestions that match individual tastes and preferences. It reduces the time required to search for suitable food options by automatically presenting relevant recommendations. In addition, it increases user satisfaction by offering tailored suggestions that improve the efficiency and convenience of the ordering process. Furthermore, it encourages users to explore new menu items based on their previous orders and preferences, thereby increasing engagement with the system.

4.1.3. Required data

This feature requires several types of data to function effectively, including users' historical order data, food preferences such as liked and disliked items, and detailed information about menu items and their categories. It also relies on popularity metrics derived from overall user orders, as well as time-based ordering patterns to better understand user behavior and improve recommendation accuracy[5],[3].



4.2. Predicting Busy Times (Demand Forecasting)

4.2.1. What it does

This feature applies artificial intelligence and predictive analytics techniques to historical order data in order to forecast peak demand periods within campus cafeterias. It analyzes patterns in ordering behavior over time to identify busy and less busy intervals, allowing it to predict when the cafeteria is likely to experience high traffic.

4.2.2. Benefits to users

This feature improves the overall user experience by helping students avoid peak hours and reduce waiting time when ordering food. It enables users to better plan their meals by choosing less crowded times, which increases convenience and efficiency. In addition, it enhances user satisfaction by minimizing congestion during busy periods and providing a smoother ordering experience. It also indirectly benefits cafeteria operations by supporting better resource allocation and staff management during predicted peak times.

4.2.3. Required data

This feature requires historical order timestamps, order volume data across different time intervals (such as hours, days, and weeks), and seasonal or periodic patterns in user activity. It may also benefit from additional contextual data such as academic calendars, exam periods, and holidays to improve the accuracy of demand forecasting.



4.3. Ethical Considerations

4.3.1. Could your AI be biased? How?

Yes, the AI system could potentially be biased if it relies heavily on historical order data that reflects repetitive or limited user behavior. For example, if the majority of students frequently order fast food, the recommendation system may over-prioritize these items and repeatedly suggest them while under-recommending healthier or less popular food options. This results in a popularity bias that reduces diversity in recommendations.

4.3.2. How will you protect user privacy?

User privacy will be protected by minimizing the collection of personal data and only storing information necessary for system functionality. Sensitive data such as order history will be encrypted and securely stored. In addition, user identities will be anonymized when used for AI processing or training, ensuring that data cannot be traced back to individual users. Access to data will also be strictly limited to authorized system components.

4.3.3. How will you make AI fair for all users?

Fairness will be ensured by designing the recommendation system to provide diverse food options rather than relying solely on popularity-based outputs. The AI model will be regularly evaluated and tested to detect and reduce bias in its predictions. Furthermore, training data will be collected from a diverse range of users to avoid over-representation of specific groups. The system will also allow users to reset or customize their preferences to ensure equal opportunity for all menu items to be recommended.



5. Sustainability & Green Software

5.1. Sustainability Overview

In software engineering, sustainability is understood as a system's capacity to endure, and it is treated as a quality concern alongside usability, security, and safety rather than as a separate afterthought [7]. It concerns the long-term environmental, social, economic, individual, and technical effects of a system and should be considered across requirements, design, implementation, testing, and deployment.

Importantly, sustainability matters even when it is not a system's primary goal: although the **Campus Food Ordering System** is built to streamline ordering rather than to address sustainability directly, its design and daily operation still produce effects across all five dimensions, a point captured by the *Karlskrona Manifesto* [7]. The analysis below examines each dimension by pairing them with a specific design choice in the system.

5.2. Five Sustainability Dimensions

5.2.1. Technical Sustainability

Technical sustainability is concerned with longevity, maintainability, and the ability to evolve without premature obsolescence. The system supports this mainly through its microservices architecture: because the users, menu, orders, payments, notifications, and AI/analytics services are independent and communicate over REST APIs, any one of them can be modified, scaled, or replaced without rewriting the rest of the system.

Containerization with Docker and the automated CI/CD pipeline supports this, since the coverage gate and vulnerability scanning keep the repository dependable across changes, reducing the technical debt that usually shorten a system's useful life [8].

5.2.2. Individual Sustainability

This dimension addresses the privacy and autonomy of the people who use and build the system. For students, advance ordering and pickup time slots remove the need to wait in long peak-hour queues, while real-time order status reduces the uncertainty of manual ordering; both lower the everyday friction around campus meals.



Individual autonomy is protected in the AI features, where user data is anonymized in processing, and users can customize their preferences, so the system supports rather than quietly shapes their choices. For the development team, the automated pipeline removes repetitive manual deployment work, contributing to a more human workload.

5.2.3. Economic Sustainability

Economic sustainability is about delivering value while keeping long-term costs viable. The microservices design allows the system to scale only the services under load, for example the orders service during the lunch peak, rather than the entire application, which avoids paying for unnecessary capacity.

The demand-forecasting feature contributes indirectly by helping the cafeteria plan inventory and staffing around predicted peaks, reducing both food waste and over-staffing costs [3].

5.2.4. Social Sustainability

Social sustainability concerns equitable access, inclusivity, and trust. The system widens access through a mobile-friendly interface and role-based accounts (student, staff, manager, administrator, and optional guest), so different groups interact with it appropriately.

Fairness is a deliberate concern in the recommendation engine: rather than amplifying only popular items, it is designed to surface diverse options so that less-ordered menu items retain a fair chance of being seen, which prevents the system from reinforcing a narrow set of choices [9].

Trust is maintained by encrypting sensitive data such as order history and restricting access to authorized components, giving users confidence in how their information is handled.



5.2.5. Environmental Sustainability

Environmental sustainability focuses on minimizing energy use, emissions, and waste across the system's lifecycle. Operationally, dynamic scaling prevents servers from being over-provisioned during off-peak hours, and the multi-stage Docker build keeps production images as small as possible, both of which reduce wasted compute and energy[8].



6. References

- [1] K. Kushwah, S. Sharma, A. Sharma, and M. Negi, "CafeEase: Improving College Cafeteria Experiences," *International Journal of Novel Research and Development (IJNRD)*, vol. 9, no. 4, Apr. 2024. [Online]. Available: <https://www.ijnrd.org/papers/IJNRD2404096.pdf>
- [2] J. N. Bondevik, K. E. Bennin, Ö. Babur, and C. Ersch, "A Systematic Review on Food Recommender Systems," *Expert Systems with Applications*, vol. 238, Article 122166, Mar. 2024. doi: 10.1016/j.eswa.2023.122166
- [3] T. Tanizaki, T. Hoshino, T. Shimmura, and T. Takenaka, "Demand Forecasting in Restaurants Using Machine Learning and Statistical Analysis," *Procedia CIRP*, vol. 79, pp. 679–683, 2019. doi: 10.1016/j.procir.2019.02.042
- [4] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
- [5] A. Ajami and B. Teimourpour, "A Food Recommender System in Academic Environments Based on Machine Learning Models," *arXiv preprint arXiv:2306.16528*, Jun. 2023.
- [6] X. Gao, F. Feng, X. He, H. Huang, X. Guan, C. Feng, Z. Ming, and T.-S. Chua, "Hierarchical Attention Network for Visually-Aware Food Recommendation," *IEEE Transactions on Multimedia*, vol. 22, no. 6, pp. 1647–1659, 2020. doi: 10.1109/TMM.2019.2945180
- [7] C. Becker, S. Betz, R. Chitchyan, L. Duboc, S. M. Easterbrook, B. Penzenstadler, N. Seyff, and C. C. Venters, "Requirements: The Key to Sustainability," *IEEE Software*, vol. 33, no. 1, pp. 56–65, 2016. doi: 10.1109/MS.2015.158
- [8] "Sustainability Design," *Sustainabilitydesign.org*, 2026. [Online]. Available: <https://sustainabilitydesign.org/>
- [9] J. Ning, J. Li, Z. Li, Y. Tian, and S. Li, "Study on the Form of Reservation of Food Orders in Campus Canteens in the Post-epidemic Era," *International Journal of Education and Humanities*, vol. 8, no. 2, pp. 96–102, 2023. doi: 10.54097/ijeh.v8i2.7754



7. Team Contribution Statement

Student Name: Nora Mohammed Alkabkabi

Individual Contribution:

In this project, I was responsible for researching and writing the System Overview section of the report. This included drafting the problem description, explaining the challenges students and cafeteria operations face on campus (long queues, manual ordering, limited visibility, and lack of operational data). I also defined the system objectives, outlining what the Campus Food Ordering System aims to achieve, such as digitizing ordering, reducing wait times, improving accuracy and transparency, and supporting secure role-based access.

Additionally, I identified and documented the main features of the system across ordering, accounts, payments, cafeteria management, administration, and AI. I also defined the users and roles (student, cafeteria staff, manager/vendor, system administrator, and optional guest), specifying the actions each role can perform within the platform.

Student Name: Jana Saad Alotaibi

Individual Contribution:

In this project, I was responsible for the Microservices Design section of the report. I designed the system architecture by decomposing the Campus Food Ordering System into six microservices, defining the responsibilities of each service, the data managed by each service, and the communication between services. I also documented the database-per-service approach and selected appropriate database technologies for each microservice.

Additionally, I designed the system architecture diagram, identified the external systems integrated with the platform, and justified the selected technology stack, including Node.js, PostgreSQL, MongoDB, REST APIs, Docker, and RabbitMQ. My contribution focused on ensuring scalability, maintainability, service independence, and efficient communication between microservices.



Student Name: Arwa Alhassani

Individual Contribution:

In this project, I was responsible for the DevOps Strategy section of the report. I designed the CI/CD pipeline and produced its diagram, wrote the pipeline overview, and explained each stage of the pipeline from the source-control trigger through to post-deployment verification. I also evaluated three CI/CD tools (GitHub Actions, GitLab CI, and Jenkins) and justified the selection of GitHub Actions, and I wrote the testing strategy covering unit testing, integration testing, and user acceptance testing.

Student Name: Jory Alkhudari

Individual Contribution:

In this project, I served as the team leader, overseeing the overall coordination of the team's work and ensuring consistency across all sections of the report.

I was also responsible for researching and writing the AI section of the report. This included explaining the proposed AI features, their benefits to users, and the data required for their implementation. I also researched and wrote the ethical considerations related to AI, including bias, privacy, and fairness, and discussed how these challenges can be addressed within the system.

In addition, I contributed to Part 2 (Microservices Design) by providing feedback, reviewing the system architecture, and supporting the refinement of design decisions to ensure consistency and alignment with the overall system requirements.



Student Name: Jana Ghazi AlHuthayli

Individual Contribution:

In this project, I was responsible for writing the Sustainability & Green Software section of this report. This meant explaining the idea of sustainability in software engineering. Basic concepts of sustainability were discussed, e.g. persistence of the system, sustainability should be considered even if it is not the main goal of the system, direct, indirect and rebound effects.

I also did some research on the five sustainability dimensions. and wrote an analysis using the five dimensions of our Campus Food Ordering System. I linked each dimension to a particular design choice in our project. I also discussed the advantages of these choices and the trade-offs that come with them.